[VIT logo - insert here]

[Course code]

[Course name]

OSmosis

An AI-Powered Operating Systems Learning Platform

Project Report

Submitted by

[Student name]

[Registration number]

Under the guidance of

[Faculty name]

Vellore Institute of Technology

[School / department]

[Academic year and semester]

Table of Contents

1. Introduction 2
2. Problem Statement 3
3. Objectives 4
4. Methodology 5
   4.1 Overall Architecture 5
   4.2 Knowledge-Base Preparation 6
   4.3 Retrieval-Augmented Generation 7
   4.4 LLM Integration 8
   4.5 Numerical Solvers 8
   4.6 Reasoning Debugger (Check my working) 10
   4.7 Predict, Run, Explain (OS Lab) 11
   4.8 Misconception Graph and Adaptive Learning 12
   4.9 Quiz and Progress Management 13
   4.10 Technology Stack 14
5. Research and Development 15
   5.1 Research and Design Decisions 15
   5.2 Prompt Engineering 15
   5.3 Execution and Implementation Workflow 16
   5.4 RAG Evaluation and Experimentation 17
   5.5 Software Verification and Testing 19
   5.6 Results and Observations 20
6. Conclusion 21
7. Future Work 22
8. References 23

If the page numbers above are missing or out of date, right-click the table and choose Update Field.

1. Introduction

An operating system manages a computer's processor, memory, storage and devices on behalf of the programs that run on it. Its central policies - which process runs next, whether granting a resource can lead to deadlock, which page leaves memory, in what order disk requests are served - are algorithms whose effects can be worked out by hand. Understanding them is a core part of a computer-science degree, both for later courses in systems and for writing software that behaves well on real machines [1].

These algorithms are hard to learn from static notes alone. A textbook or a slide deck shows one finished Gantt chart or one safe sequence, but not the decision made at each instant, the state that justified it, or the point at which a student's own working diverged from the correct one. CPU scheduling, deadlock avoidance with the Banker's algorithm, memory management, page replacement and disk scheduling are particularly suited to interactive explanation, because each is a sequence of small, checkable decisions.

Large language models (LLMs) can explain such topics, but a general model answers from its training data rather than from the course's material, and it may not follow the conventions a particular course uses. Retrieval-augmented generation (RAG) addresses this by retrieving relevant passages from a document collection and asking the model to answer from them [2]. Grounding answers in the course's textbook and slides makes them traceable to a page or slide.

OSmosis was built for the Operating Systems course on this basis, but it is not only a chatbot. It combines four parts: (i) a RAG tutor that answers from the course textbook, the teacher's slides and the lab notes, with citations and figures; (ii) exact solvers that compute numerical problems in deterministic Python code; (iii) learning features that check a student's own working step by step, let the student predict and replay an algorithm's decisions, and record which misconceptions recur; and (iv) an adaptive quiz. It runs locally as a Streamlit web application.

Scope of the completed system. Course-grounded question answering over 2,222 passages and 928 images; exact solvers for CPU scheduling (FCFS, SJF, SRTF, priority, Round Robin), the Banker's algorithm, page replacement (FIFO, LRU, Optimal) and disk scheduling (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK); a Reasoning Debugger, an interactive OS Lab and a Misconception Graph for all four solver families; and a quiz with multiple-choice, viva and numerical questions. The system uses an existing instruction-tuned LLM; no model is trained or fine-tuned.

2. Problem Statement

Students of operating systems must learn both concepts (what a race condition is, why a state is unsafe) and procedures (how to compute waiting times under SRTF, how to find a safe sequence). Several difficulties arise when the available resources are a textbook, lecture slides and general-purpose AI assistants:

- **Static material shows results, not reasoning.** Worked examples show a final schedule or table, but not why each decision was taken at the moment it was taken.
- **Numerical problems are error-prone.** A single wrong decision - a missed preemption, a Round Robin queue in the wrong order - changes every value that follows, and a student who compares only the final answers cannot tell where the working first went wrong.
- **Generic AI answers are not tied to the course.** An assistant that is not grounded in the course material may use different conventions (for example, whether a larger priority number means higher priority, or how a process arriving at the instant a quantum expires is queued) and does not cite where its answer comes from. Its arithmetic in numerical problems cannot be assumed to be correct.
- **Little interactivity or personalisation.** Static resources do not let a student predict and test an algorithm's next step, and do not track which mistakes a particular student keeps making.

The problem addressed by OSmosis is therefore: **to provide Operating Systems students with explanations grounded in their own course material, exact solutions to numerical problems, and feedback that locates the specific step where their own reasoning goes wrong, together with interactive practice that responds to the mistakes they actually make.**

3. Objectives

The objectives of the project, and their status in the completed system, are:

1. **Course-grounded question answering.** Answer OS questions from the course textbook, teacher slides and lab notes using RAG, and say so when the material does not cover a question. *Achieved.*
2. **Citations and visual material.** Cite the page or slide behind every factual sentence and show the relevant textbook figures or slide images. *Achieved;* figure selection depends on retrieval and is weaker for some questions (Section 5.6).
3. **Exact numerical solving.** Solve CPU-scheduling, Banker's-algorithm, page-replacement and disk-scheduling problems deterministically, with every step shown. *Achieved and tested against the course's worked examples.*
4. **Step-by-step diagnosis of student reasoning.** Compare a student's own working with the exact solution, find the first wrong step and give staged feedback. *Achieved for all four solver families.*
5. **Interactive algorithm execution.** Let students predict, run and inspect each decision of a real algorithm run. *Achieved for all four solver families.*
6. **Misconception tracking and targeted practice.** Record evidence of mistakes, distinguish tentative from recurring problems and recommend practice. *Implemented and unit-tested;* the rules have not been validated with students.
7. **Quizzes, viva questions and adaptive learning.** Generate and mark questions, with difficulty that adapts per topic. *Achieved;* the quality of LLM-written questions was not evaluated quantitatively.
8. **Evaluation.** Measure retrieval quality on a labelled question set and verify software correctness with automated tests. *Achieved* (78-question retrieval evaluation; 71 automated tests).

4. Methodology

4.1 Overall Architecture

OSmosis is a single Python application. The user interface is a Streamlit app (`app.py`) with six pages: Chat, Numerical solver, Check my working, OS Lab, Quiz and My learning map. Application logic lives in the `rag` package. A question first passes through a numerical detector: if it states a numerical problem, the matching solver computes the exact answer. In parallel, the retrieval subsystem selects the six most relevant passages, and the LLM writes an explanation grounded in them, with the solver's result supplied as fixed facts. The learning features use the solvers' structured traces, and the evidence they record is stored in a local JSON progress file. Figure 1 shows the data flow as implemented.

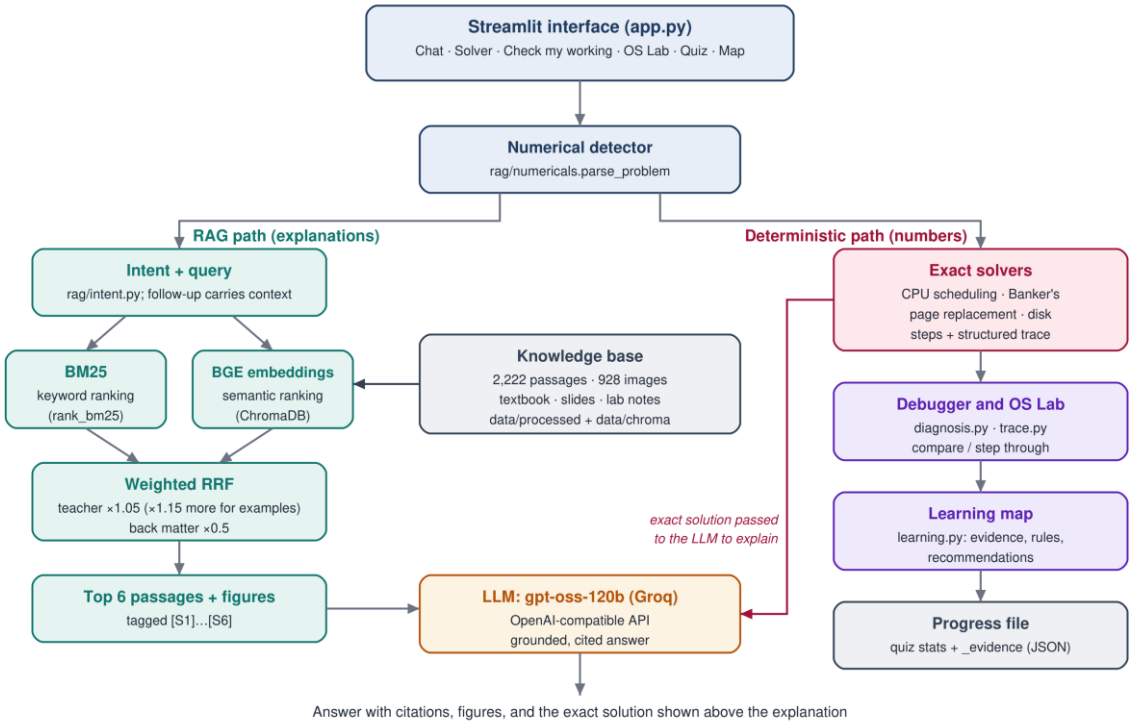


Figure 1: *System architecture of OSmosis as implemented. The left branch produces explanations from retrieved course passages; the right branch produces exact numerical results, which are passed to the LLM only to be explained.*

Table 1: Main components and their source files

Component	Source	Responsibility
User interface	app.py	Six Streamlit pages, solution rendering, charts, staged feedback
Ingestion	rag/ingest.py, parse_book.py, parse_slides.py, parse_notes.py	Course files to passages, figures and slide images
Indexing	rag/build_index.py, embeddings.py	BGE embeddings (cached) stored in ChromaDB

Component	Source	Responsibility
Retrieval	rag/retriever.py, intent.py, config.py	BM25 + embedding ranking, weighted fusion, figure linking
Answering	rag/ask.py	Prompt construction, LLM call, citation clean-up, follow-ups
Solvers	rag/solver.py, banker.py, paging.py, disk.py, numericals.py	Exact numerical solutions and structured traces
Learning features	rag/diagnosis.py, trace.py, misconceptions.py, learning.py	Debugger, OS Lab, evidence and misconception graph
Quiz	rag/quiz.py	Question generation, marking, adaptive levels, progress file

4.2 Knowledge-Base Preparation

The knowledge base is built from the course files by `python -m rag.ingest`, with a parser for each kind of source (Figure 2):

- **Textbook** (Silberschatz, Galvin and Gagne, 10th edition [1]; chapters 2-11, 13-15 and 20). `parse_book.py` uses PyMuPDF [13] to read the PDF, tags each paragraph with its section number, crops figures and the Gantt charts printed in the text, and links each passage to the figures it mentions (for example, "Figure 5.5").
- **Teacher slides** (Modules 1-5). `parse_slides.py` groups consecutive slides that share a title (a worked example spread over several slides), packs them into passages and renders every slide as an image. Slides that are mostly pictures but hold worked examples found nowhere else were transcribed by hand into JSON files (39 slides).
- **Lab material.** A Linux-commands handout (PDF), shell-programming slides (PPTX, read with python-pptx [14]) and a bash exercise sheet in the legacy Word 97 format, based on material by H.-W. Loidl [16]. `parse_notes.py` splits these documents at their section headings and keeps line breaks and indentation, so shell scripts remain readable; the .doc file is read directly without Microsoft Word.

Passages are packed to a target of about 1,100 characters (maximum 1,600). Each passage keeps metadata: source identifier, kind (textbook or teacher material), section title, page or slide range, and linked figure identifiers, from which citations such as "Silberschatz 10e, 5.3.1 First-Come, First-Served Scheduling, pp. 206-207" are generated. `python -m rag.build_index` embeds every passage and figure caption with BAAI/bge-base-en-v1.5 [5], [6] through the sentence-transformers library [7] and stores the vectors in ChromaDB [8] with cosine distance. Embeddings are cached by a hash of the text, so adding material only embeds new passages.

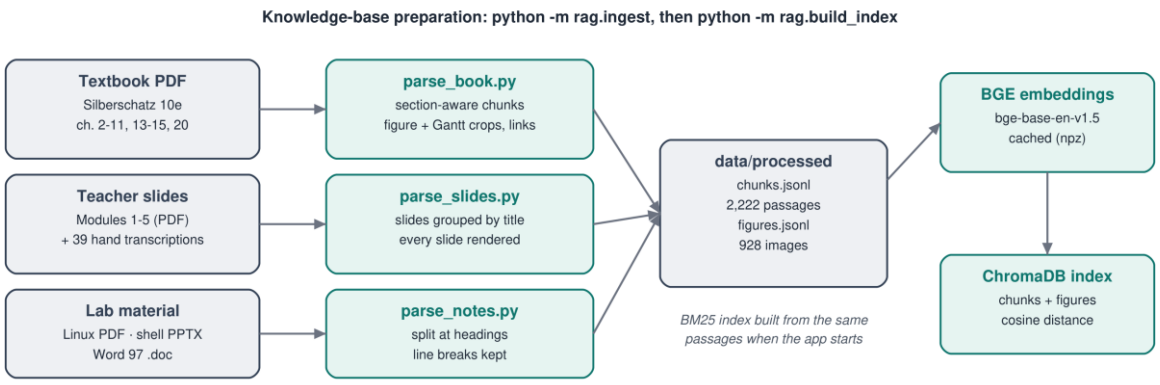


Figure 2: *Knowledge-base preparation pipeline.*

Table 2: Knowledge-base composition (current index)

Source	Passages	Images
Silberschatz 10e (textbook)	1,783	297 figures and Gantt charts
Teacher slides, Modules 1-5	346	629 slide images
Lab notes and exercises (Linux, shell)	93	2 slide diagrams
Total	2,222	928

4.3 Retrieval-Augmented Generation

Query processing. The question is first classified by a small rule-based intent classifier (`rag/intent.py`) as numerical, example, comparison, definition, follow-up or conceptual. A short question that refers back to the previous turn ("why does it happen?") is retrieved together with the previous question, so the follow-up keeps its topic; the last three turns are also passed to the LLM.

Keyword retrieval. BM25 [3], implemented with the rank_bm25 library [4], ranks passages by term overlap. Queries and passages are tokenised the same way, stop words are removed and common abbreviations are expanded (for example, SRTF to "shortest remaining time first").

Semantic retrieval. The query is embedded with BGE, using the query instruction the model card recommends for retrieval [6], and the 50 nearest passages are taken from ChromaDB.

Fusion and ranking. The two rankings (top 50 each) are combined with weighted Reciprocal Rank Fusion (RRF) [9]. A passage *d* receives

$$score(d) = \Sigma_r \, w_r / (k + rank_r(d)), \quad k = 60, \quad w_{BM25} = w_{BGE} = 1$$

and the score is then multiplied by 1.05 for teacher material, by a further 1.15 when the question asks for an example or a numerical (the course's worked examples are in the slides), and by 0.5 for textbook back matter (exercises, programming projects, bibliography and further reading),

which names topics without explaining them. These settings were selected experimentally (Section 5.4). Passages whose first 120 characters repeat an earlier passage are dropped.

Context construction and citations. The top six passages are numbered [S1]-[S6] in the prompt, each preceded by its citation. Figures linked to the top two passages, plus figures that rank in the top ten of both rankers in a separate figure search, are attached (at most four, alternating textbook and teacher figures). After the LLM answers, any citation tag that does not correspond to a supplied passage is removed, and if the answer cites nothing the source list is hidden, because the model has stated that the material does not cover the question.

Teacher conventions. The prompt instructs the model to prefer the teacher's slides where they differ from the textbook and lists the course's scheduling conventions (for example, smaller priority number = higher priority; on a tie, the earlier arrival and then the lower process number). The small boost means teacher material is preferred only when it is also relevant; an unrelated slide can no longer outrank a passage that both rankers place first (Section 5.4).

4.4 LLM Integration

Answers are written by `openai/gpt-oss-120b` [10] served by Groq [11] through its OpenAI-compatible chat-completions API. The base URL, API key and model name are read from the environment variables LLM_BASE_URL, LLM_API_KEY and LLM_MODEL, so any OpenAI-compatible endpoint can be used, including a local Ollama server. The key is never stored in the project. Requests use temperature 0 for answers (0.7 for question generation), a 120-second timeout and up to three retries on rate limiting.

Fallback behaviour. Without an API key, or if the API fails, the chat shows the retrieved passages and figures instead of a written answer, and every feature that does not need the LLM keeps working: the numerical solvers, the Reasoning Debugger, OS Lab, the learning map and the quiz's numerical and built-in questions.

Insufficient evidence. The system prompt requires the model to answer only from the supplied sources and to say so and stop when they do not contain the answer, rather than fill the gap from its own knowledge.

4.5 Numerical Solvers

Numerical problems are solved by deterministic Python code, not by the LLM. A question in natural language or as a pasted table is parsed into a problem object; the solver records every decision both as text and as structured data. Table 3 lists the supported algorithms.

Table 3: Exact solvers

Solver	Algorithms	Inputs and outputs
CPU scheduling (solver.py)	FCFS, SJF, SRTF, priority (preemptive and non-preemptive), Round Robin; optional I/O bursts	Arrival and burst times, priorities, quantum; Gantt chart, CT, TAT, WT, RT, averages, utilisation and every decision
Banker's algorithm (banker.py)	Need matrix, safety algorithm, resource-request algorithm, deadlock detection	Allocation, Max or Need, Available or total instances, optional request; safe sequence with Work after each step, request verdict
Page replacement (paging.py)	FIFO, LRU, Optimal; several frame counts (Belady's anomaly)	Reference string and frames; frame table, hits, faults, fault rate, victim at each step
Disk scheduling (disk.py)	FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK	Request queue, head position, direction, disk size; service order, head path, total movement (return seek shown separately)

The solvers follow the course conventions: TAT = CT - AT, WT = TAT - BT (with I/O, BT is the total CPU burst, as in the teacher's example), RT = first time on the CPU - AT; in Round Robin, a process that arrives at the instant a quantum expires joins the ready queue before the preempted process. The Banker's safety algorithm checks the processes in table order, pass after pass, which reproduces the slides' sequences; other valid safe sequences are accepted by the debugger. In the chat, "Explain SRTF with an example" (or the Banker's algorithm, LRU, C-SCAN) solves one of 20 worked examples from the slides and textbook in code before the LLM explains it (Figure 3).



Figure 3: *Chat answer to "Explain SRTF with an example" (application screenshot, without an LLM key): the teacher's worked example with an I/O burst (Module 3, slides 101-114) solved by the scheduler, with the Gantt chart, the CT/TAT/WT/RT table and averages. The retrieved slide passages follow below it.*

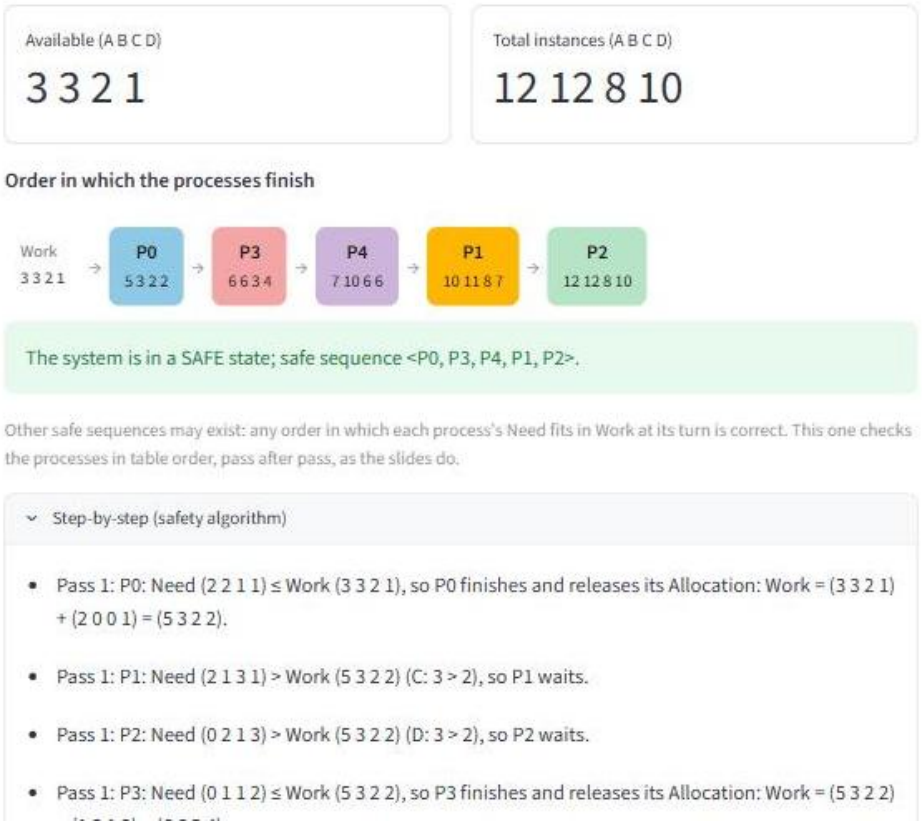


Figure 4: *Numerical solver, Banker's algorithm (application screenshot): the order in which processes finish with Work after each step, the verdict, and the first steps of the safety algorithm, for the teacher's Example 2.*

4.6 Reasoning Debugger (Check my working)

The student enters their own working: a Gantt chart and CT/TAT/WT/RT table, a safe sequence and verdicts, hit/fault marks, or a disk service order and total. `rag/diagnosis.py` compares it with the solver's reference solution, deterministically and without the LLM:

- **Earliest verifiable error.** Both Gantt charts are expanded to one entry per time unit; the first differing instant is the first error. The solver's recorded state at that instant (ready queue, remaining bursts, arrivals, preemptions) is used to classify it: a process run before it arrived or during its I/O, a missed or unjustified preemption, a Round Robin queue-order error, or a wrong selection by the algorithm's rule.
- **Causes only on evidence.** A misconception is named only when the answer shows its defining pattern: the whole chart equals another algorithm's schedule; a waiting time equals the response time; a completion time equals the end of an earlier slice; the whole chart follows the opposite Round Robin tie rule. Otherwise the error is reported without a cause, or the cause is marked as likely rather than certain.
- **Consequences are not counted twice.** Table values are checked against the student's own chart and own earlier values, so a CT that follows from a wrong chart is listed as a consequence of the first error, not as a new one.
- **Staged feedback.** Each finding has an observation, a hint, an explanation and a correction, revealed one at a time (Figure 5); the observation and hint never contain the

correct value. Unreadable answers are not diagnosed; the student is shown the expected format.

The same interface covers the Banker's algorithm (any valid safe sequence is accepted; the first process whose Need exceeds Work is reported), page replacement (the first wrong hit/fault mark; marks that match another algorithm's) and disk scheduling (the first wrong request; a total that omits the C-SCAN return seek). With an API key, "Explain with the course notes" adds an LLM explanation grounded in retrieved passages; the solver's solution and the diagnosis are passed to it as fixed facts, and the page labels which parts are deterministic.

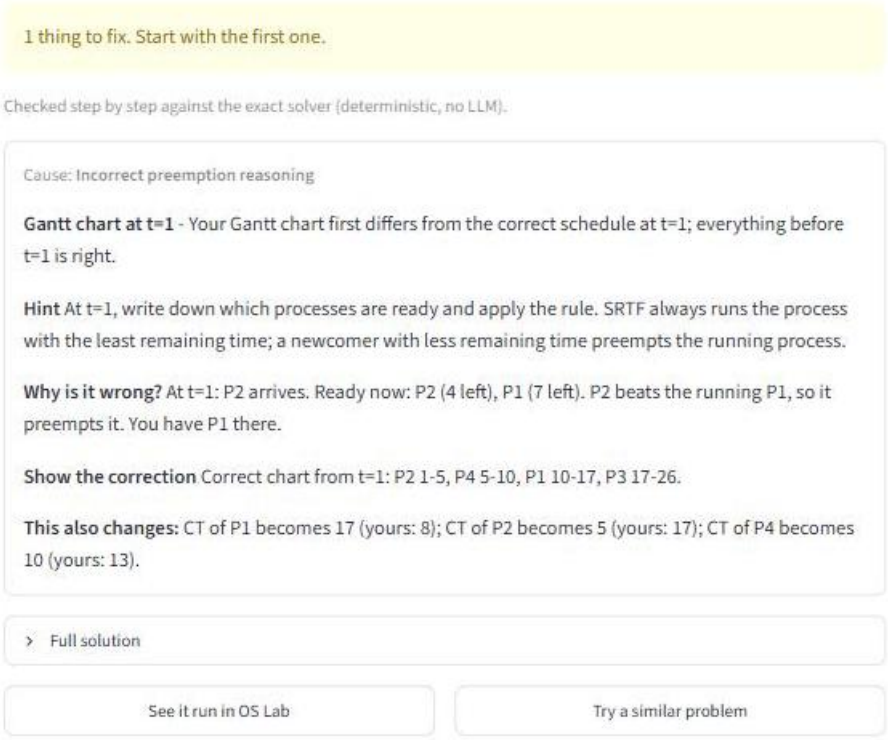


Figure 5: *Check my working (application screenshot): a student's SRTF chart that misses the preemption at t = 1, with the cause, hint, explanation, correction and the completion times that change as a result.*

4.7 Predict, Run, Explain (OS Lab)

`rag/trace.py` turns a real solver run into a sequence of events. For CPU scheduling, each event is a decision point recorded by the scheduler: the time, the process that was running, the arrivals, completions and quantum expiries at that instant, the ready queue in order, the remaining bursts and the Gantt chart so far. The student predicts which process runs next, presses Run, and sees the actual decision with the solver's own explanation and the updated ready queue (Figure 6). Previous, Next, Replay and Reset are available, the page keeps a prediction score, and the run ends with the full solution and a reflection question. The same mechanism covers the Banker's algorithm (can this process finish with the current Work?), page replacement (hit, or which page is replaced?) and disk scheduling (which cylinder next?). From the debugger, "See it run in OS Lab" opens the lab at the step where the student's working went wrong.

Because every event is read from the solver's recorded state rather than generated, the displayed trace cannot disagree with the solution; the tests replay every lab and check that the result equals the solver's (Section 5.5).

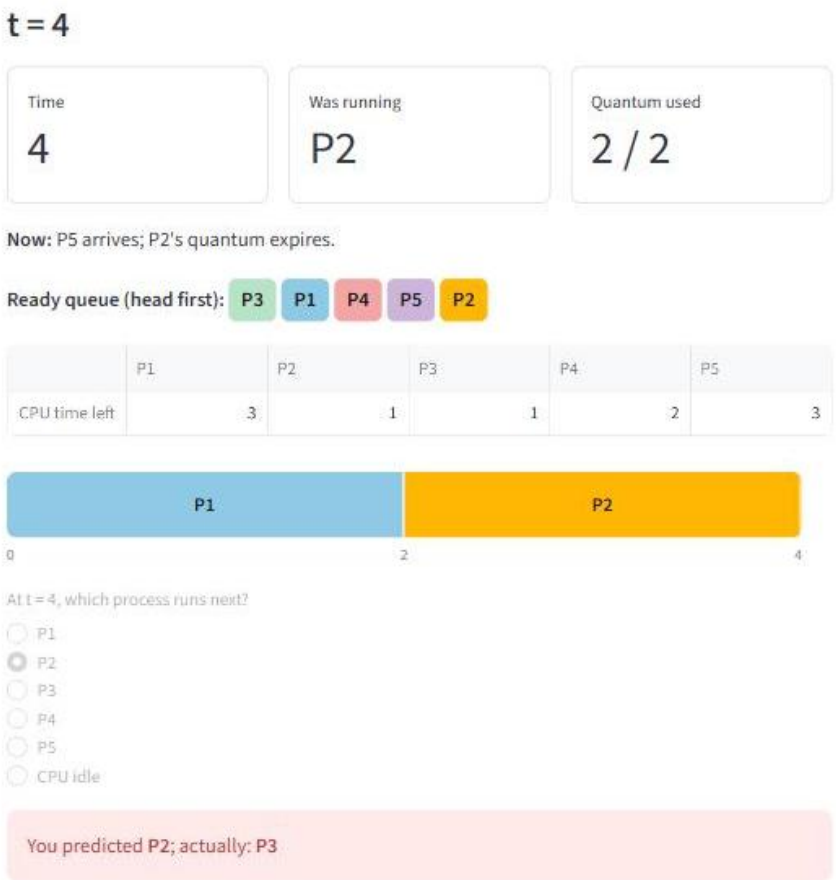


Figure 6: *OS Lab, Round Robin with quantum 2 (teacher's Example 2), at t = 4 (application screenshot): P2's quantum expires as P5 arrives; the course rule queues P5 before P2, so P3 runs next. The student had predicted P2.*

4.8 Misconception Graph and Adaptive Learning

`rag/misconceptions.py` defines 20 concepts (for example, "Round Robin and the ready queue", "Safety algorithm") with prerequisite links and the course slides that explain them, and 17 misconceptions across the four solver families, each with a stable identifier, the concept it concerns, the evidence that shows it, a remediation and a suitable practice activity.

Evidence comes from three places: debugger findings (certain or uncertain, as the validator decided), OS Lab predictions (a wrong prediction is always uncertain evidence) and quiz answers. Records are stored in the existing progress file under an `_evidence` key, so files written before this feature still load. `rag/learning.py` applies transparent rules: a misconception is *tentative* when seen once or only as a likely cause, *recurring* when seen with certainty in two or more different problems, and *resolved* after two correct attempts on the same concepts; a concept is *understood* after at least three attempts with 75% correct and no active misconception. These thresholds are deliberately simple and are not a validated mastery model.

The learning map draws concepts, prerequisites and the student's misconceptions as a graph, lists recommendations (recurring problems first, with the slides to revise and any prerequisite not yet understood), shows the evidence behind each misconception with a course passage, and links to a fresh practice problem or a quiz on the topic. Figure 7 shows the map produced by two debugger attempts and one OS Lab run during testing.

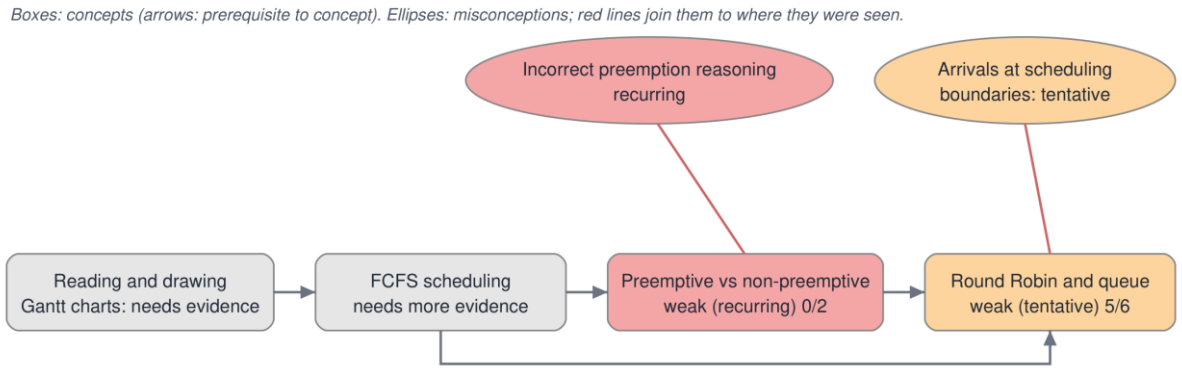


Figure 7: *Learning map produced by the application during a demonstration session (two debugger attempts with a missed SRTF preemption, one OS Lab run on Round Robin). Redrawn from the application's graph in a two-row layout for legibility; nodes, statuses and links are unchanged.*

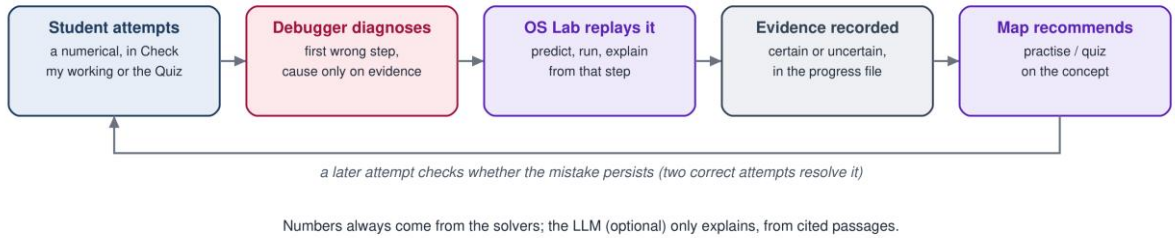


Figure 8: *The learning loop connecting the debugger, OS Lab, the evidence log and the learning map.*

4.9 Quiz and Progress Management

The quiz offers three kinds of question across 22 topics. **Multiple-choice and viva questions** are written by the LLM from four passages retrieved for the topic and returned as JSON; malformed replies fall back to a built-in bank of 24 questions. **Viva answers** are marked 0-2 by the LLM against the key points and source passage, with written feedback. **Numerical questions** for seven topics (FCFS, SJF/SRTF, priority, Round Robin, Banker's algorithm, page replacement, disk scheduling) are generated randomly and their answer keys computed by the solvers, so the key is correct by construction; some are computed multiple choice, such as "which of these is a safe sequence?", whose wrong options are verified to be unsafe.

Each topic has a level from 1 to 3: two correct answers in a row raise it, a wrong answer lowers it, and weaker or untried topics are chosen more often in adaptive mode. Progress (per-topic statistics and the evidence log) is saved to `data/quiz_progress.json`.

4.10 Technology Stack

Table 4: Technology stack (versions installed in the project environment)

Technology	Version	Role
Python	3.12.4	Application language
Streamlit [12]	1.65.0	Web interface
PyMuPDF [13] / python-pptx [14]	1.28.2 / 1.0.2	PDF and PowerPoint extraction, slide rendering
rank_bm25 [4]	0.2.2 or later	BM25 keyword ranking
sentence-transformers [7]	6.1.0	Runs the BGE embedding model
BAAI/bge-base-en-v1.5 [5], [6]	768-dimensional	Passage and query embeddings
ChromaDB [8]	1.5.9	Persistent vector store (cosine distance)
gpt-oss-120b [10] on Groq [11]	API	Answer, question and feedback generation
pandas	3.0.6	Tables in the interface

5. Research and Development

5.1 Research and Design Decisions

This subsection records the reasons for the main decisions. Which of them were tested experimentally is stated in Sections 5.4-5.6.

- **RAG for course grounding.** The tutor must answer from this course's material and show where an answer comes from. Retrieval supplies the relevant passages at question time and makes every claim citable [2]; adding material only requires re-indexing.
- **An existing instruction-tuned LLM, no training.** Training or fine-tuning a model needs data and compute beyond a course project, and grounding is provided by retrieval rather than by the model's weights. An open-weight model behind an OpenAI-compatible API also allows a local model to be substituted.
- **Deterministic solvers for numbers.** Numerical answers must be exact, and every intermediate state must be available for teaching. Code gives both; the LLM is used only to explain.
- **Diagnosis and interaction.** Knowing the correct answer is less useful to a student than knowing which step of their own reasoning failed and why. The solvers' recorded states make it possible to locate that step and to let the student test predictions against the real algorithm.
- **Hybrid retrieval.** Exact terms ("SRTF", "convoy effect") favour keyword search, while paraphrased questions favour embeddings; combining both with rank fusion avoids depending on either alone [3], [9].
- **Local Python stack.** Streamlit, ChromaDB and sentence-transformers run on a laptop without a server or database setup, which suits a local academic project.

5.2 Prompt Engineering

Four kinds of prompt are used in the application. The excerpts below are quoted from the source code (`rag/ask.py` and `rag/quiz.py`); long prompts are shortened with ellipses.

Source-grounded answering (system prompt, rag/ask.py)

```
1. Answer ONLY from the numbered SOURCES. If they do not contain the answer, say so
   and stop; do not fill the gap from your own knowledge.
2. Cite every factual sentence with its source tag, e.g. [S2]. Prefer the teacher's
   slides when they and the textbook differ, and point out the difference.
   ...
   - If a COMPUTED SOLUTION follows the SOURCES, it comes from the course's exact
     solver ... do not repeat its tables and do not recompute anything. Walk through
     its "Decisions" or "Steps" lines in order ... quote its numbers unchanged ...
7. Keep it exam-oriented: short paragraphs, key terms in bold, end with one common
   mistake to avoid.
```

The prompt also lists the course's scheduling conventions, forbids invented citation tags, and tells the model to use earlier turns only to interpret follow-up questions. The retrieved passages

are supplied in the user message under a SOURCES heading, each as `[S#] (citation)` followed by its text, then any COMPUTED SOLUTION. Constraint is reinforced in code: invalid tags are stripped, and an answer that cites nothing is shown without a source list.

Quiz and viva generation (rag/quiz.py)

```
You write exam questions for an Operating Systems course. Use ONLY the numbered
SOURCES; every question must be answerable from them. Reply with one JSON object
and nothing else.
```

The user message adds the topic, the difficulty level ("easy: recall one definition or fact" ... "hard: reason about a scenario"), the most recent questions to avoid repeating, and the required JSON keys. Replies are validated (four distinct options, one valid answer index) and rejected otherwise.

Viva marking (rag/quiz.py)

```
You mark a student's short oral-exam answer for an Operating Systems course. Judge
only against the KEY POINTS and the SOURCE. Be fair: accept correct paraphrases, do
not reward padding. Reply with one JSON object and nothing else.
```

Explaining a diagnosed mistake (rag/ask.py, explain_mistake)

```
Explain to the student why this step is wrong and which rule from the course
applies, citing the SOURCES. Use only the numbers in the COMPUTED SOLUTION and the
mistake description.
```

Here the retrieval query is the algorithm name plus the hint (which contains the rule but not the answer), and the context contains the retrieved passages, the solver's exact solution and the debugger's description of the mistake. Numerical truth is therefore always produced by code and passed to the LLM as fixed text; the LLM only explains it.

5.3 Execution and Implementation Workflow

The project developed in stages; earlier prototypes are summarised for context only.

1. **Requirement analysis and prototype.** The project began from a problem statement proposing an OS-specific LLM tutor (RAG with an existing model). An early prototype used TF-IDF retrieval over hand-written notes and covered CPU scheduling only.
2. **Knowledge-base collection and preprocessing.** The prototype was rebuilt as OSmosis around the actual course material: the Silberschatz textbook and the teacher's slides, first for Module 3 and later for Modules 1-5, with dedicated parsers, figure extraction and hand transcription of image-only slides. Linux and shell lab material was added later.
3. **Embedding and indexing.** BGE embeddings in ChromaDB, with an embedding cache so that new material can be added without re-embedding everything.
4. **Retrieval implementation and testing.** Hybrid BM25 + embedding retrieval with rank fusion and a 78-question labelled evaluation set.

5. **Numerical solver integration.** The CPU-scheduling solver, checked against the teacher's 15 worked examples, then the Banker's, page-replacement and disk solvers, checked against the slides and textbook.
6. **Chat and quiz integration.** Follow-up handling, worked examples solved in code, and the adaptive quiz with computed answer keys.
7. **Retrieval audit and tuning.** A reproducible evaluation, controlled fusion experiments and the selected configuration (Section 5.4).
8. **Educational features.** Structured solver traces, the Reasoning Debugger, OS Lab and the Misconception Graph, each implemented and tested before the next.
9. **Testing, debugging and integration.** Automated test suites for every module, browser checks of every page, and fixes for defects found (for example, a Round Robin tie-rule simulation, a graph node-identifier collision, and correct table values being reported as second mistakes).

5.4 RAG Evaluation and Experimentation

Dataset. 78 questions written for the course, each labelled with where its answer lives: textbook section numbers and/or slide, page or section ranges of the teacher and lab material. A retrieved passage is relevant if it comes from a labelled location. The questions are grouped as CPU scheduling and processes (30), newer topics (28) and Linux and shell (20), and assigned to seven topics from their labels.

Metrics. Hit@k is the share of questions with a relevant passage among the first k of the six passages given to the LLM (k = 1, 3, 6); MRR is the mean of 1/rank of the first relevant passage (0 if none in the top six).

Procedure. Each question is ranked once by BM25 and once by the embeddings; every variant is scored on the same two rankings, so variants differ only in how the rankings are combined. Every fourth question of each group (19 in all) was held out. A grid of 147 fusion settings was scored on the remaining 59; settings that lowered Hit@3 or Hit@6 there were excluded, and the best MRR was chosen (ties: Hit@1, then the setting closest to the baseline). The held-out questions played no part in the choice. The evaluation is reproduced with `python eval_retrieval.py --tune`.

Findings that guided the change. The audit confirmed that the original ×1.15 teacher boost let an unrelated teacher slide ranked 3rd and 6th outrank a textbook passage ranked 1st by both rankers (half of the scheduling questions where the hybrid lost rank 1 to the embedding ranker), and that end-of-chapter material sometimes reached rank 1. Passage identifiers in the BM25 and vector indexes were consistent (2,222 in each).

Table 5: Retrieval variants on all 78 questions

Variant	Hit@1	Hit@3	Hit@6	MRR
BM25 only	51.3%	83.3%	91.0%	0.674
Embeddings only	78.2%	89.7%	93.6%	0.842
Baseline hybrid (RRF 1:1, teacher ×1.15)	71.8%	93.6%	98.7%	0.830
Embedding-heavy (BM25 weight 0.5)	75.6%	94.9%	97.4%	0.843
BM25-heavy (embedding weight 0.5)	71.8%	94.9%	98.7%	0.825
Baseline without teacher boost	71.8%	87.2%	98.7%	0.817
Baseline, back matter ×0.5	74.4%	94.9%	98.7%	0.846
Selected configuration	76.9%	94.9%	98.7%	0.858

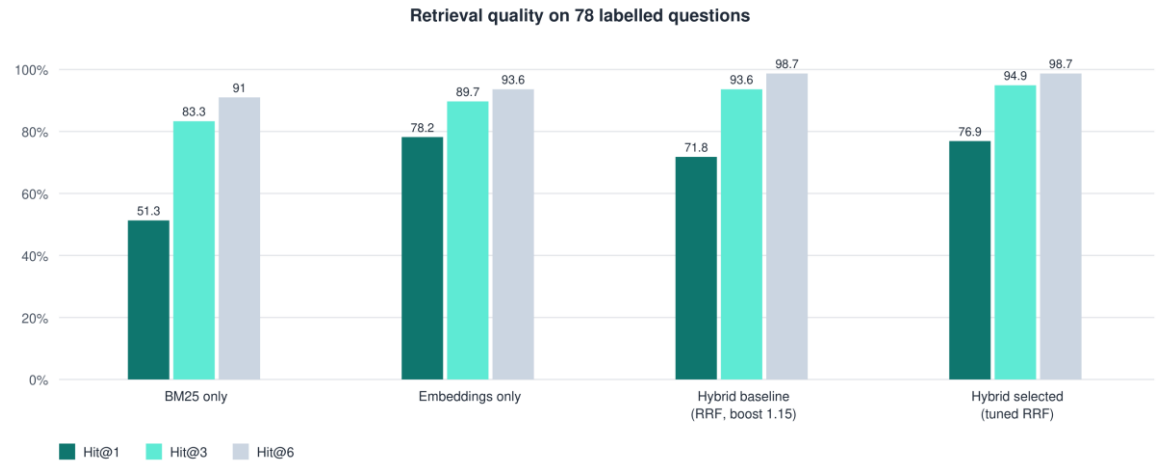


Figure 9: *Retrieval quality of the main variants (78 questions).*

Table 6: Held-out validation (19 questions not used for tuning)

Configuration	Hit@1	Hit@3	Hit@6	MRR
Baseline hybrid	84.2%	100%	100%	0.921
Selected configuration	94.7%	100%	100%	0.965

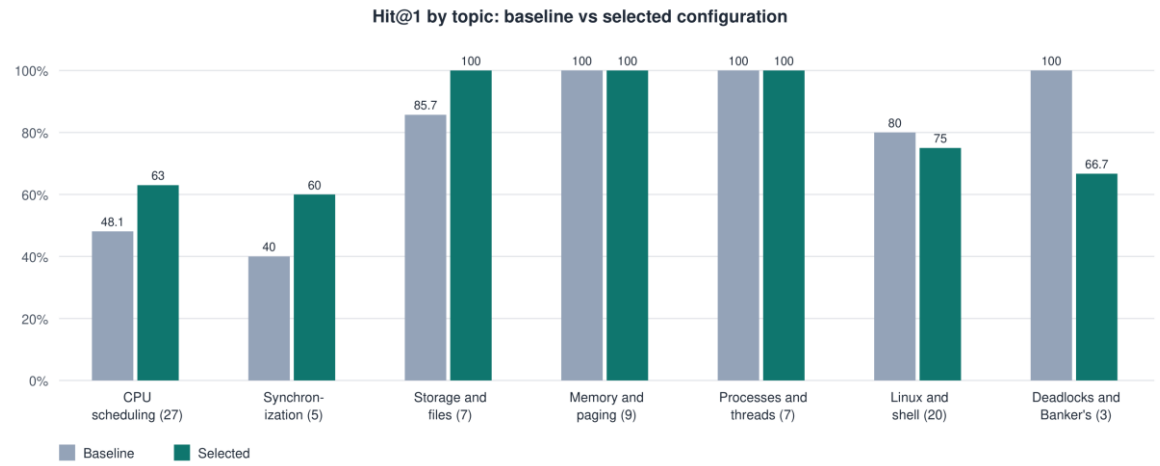


Figure 10: *Hit@1 by topic before and after tuning (number of questions in brackets).*

Selected configuration. Equal weights, k = 60, teacher boost 1.05, a further ×1.15 for example and numerical questions, and back matter ×0.5 (Section 4.3). It improves every overall metric and the held-out results. CPU-scheduling Hit@1 rose from 48.1% to 63.0%, but did not reach the embedding-only figure (77.8%); most remaining misses place an adjacent section or slide range first and the labelled one second. Linux and shell and the deadlock topic each lost one question at rank 1.

Limitations. 78 questions are enough to compare configurations but not to claim statistical significance; for the small topics (three and five questions) one question changes Hit@1 by 20-33 points. The labels were written by the developer. One question is not answered in the top six ("What happens to a process when its time slice ends?"), and "What is the convoy effect?" ranks its answer third, because the embedding model does not place the right passages in its top 50.

5.5 Software Verification and Testing

Seven automated test suites (71 tests) were executed on the final code; all passed. None needs an API key.

Table 7: Automated test suites (all executed and passing)

Suite	Tests	What is verified
test_solver	4	Scheduling solver reproduces the teacher's 15 worked examples; parsing of question formats; missing details are reported
test_numericals	13	Banker's, page-replacement and disk solvers against the slides and textbook (e.g. FIFO/LRU/Optimal 15/12/9 faults; the textbook disk exercise); input layouts; routing between solvers; follow-ups
test_quiz	12	Computed answer keys, built-in bank, adaptive levels, LLM reply validation (with a stubbed LLM), worked-example selection
test_retrieval	11	Weighted RRF arithmetic and determinism, teacher boost and intent, duplicates, metrics, citation tags, follow-up queries, figure links

Suite	Tests	What is verified
test_diagnosis	15	Debugger accepts correct work, finds the first wrong step, names causes only on evidence, does not double-count consequences, hides answers in hints, all four solvers
test_trace	8	Every OS Lab run replays to the solver's exact result; event order; session controls (predict, run, next, replay, reset, jump)
test_learning	8	Evidence rules (uncertain never confirmed, repeats recur, correct work resolves), old/empty/invalid progress files, the graph

In addition, every page of the application was exercised in a browser without an API key: chat retrieval with citations and figures, all four solver forms, the debugger, OS Lab, the learning map and the offline quiz. The retrieval evaluation (Section 5.4) is a separate, reproducible measurement.

Not verified in this round: answers, questions and viva marks written by the LLM, and the "Explain with the course notes" explanation, because no API key was available in the test environment; no study with students was carried out.

5.6 Results and Observations

- **Exact numerical results.** All four solvers reproduce the course's and textbook's worked examples, and quiz answer keys are computed rather than generated.
- **Grounded answering with measured retrieval.** The six passages given to the LLM contain a relevant one for 98.7% of the 78 labelled questions, and the first passage is relevant for 76.9% (94.7% on the held-out questions).
- **Diagnosis beyond the final answer.** The debugger locates the first wrong step and its cause in the test cases for missed preemptions, Round Robin queue errors, arrival handling, I/O bursts, formula confusion and wrong algorithms, while hints withhold the answer.
- **Trace fidelity.** OS Lab events are read from the solvers' recorded states, and every lab replays exactly to the solver's result.

Main limitations. CPU-scheduling retrieval still ranks a neighbouring section first for about a third of those questions; figures are attached only from the top two passages, so a weak first rank also loses the figures (for example, the convoy-effect question); the learning-map thresholds are heuristic and unvalidated; LLM-generated content was not evaluated quantitatively; and during browser testing the progress file once held fewer evidence records than the session had displayed, which suggests that concurrent browser sessions can overwrite each other's evidence. This has not yet been diagnosed.

6. Conclusion

OSmosis addresses a specific difficulty in learning operating systems: students need explanations tied to their own course, exact solutions to numerical problems, and feedback that shows where their own reasoning fails. The implemented system answers from the course textbook, the teacher's slides and the lab notes through hybrid retrieval, with citations to pages and slides, and leaves every number to deterministic solvers for CPU scheduling, the Banker's algorithm, page replacement and disk scheduling.

Separating grounded explanation from exact computation is the central design choice. It allows the LLM to be used where it is strong - explaining in the course's terms - while keeping every Gantt chart, safe sequence and fault count correct by construction, and it makes the system diagnostic: the same recorded solver states let the Reasoning Debugger locate the first wrong step in a student's working, let OS Lab replay the algorithm's actual decisions, and let the learning map record which misconceptions recur.

The main achievements are supported by measurement: the solvers reproduce the course's worked examples; a reproducible retrieval evaluation on 78 labelled questions improved first-rank accuracy from 71.8% to 76.9% (84.2% to 94.7% on held-out questions) without reducing top-3 or top-6 accuracy; and 71 automated tests pass. Validation with students, and a quantitative evaluation of the LLM-generated content, remain to be done.

7. Future Work

- **Larger and independently labelled retrieval evaluation**, including questions written by students, to make the per-topic results reliable.
- **A reranking stage** (for example, a cross-encoder over the top fused candidates) to separate adjacent sections, the main remaining retrieval error, and a more robust choice of figures.
- **More course material**: past question papers and additional teacher notes, which the ingestion pipeline already supports.
- **Lab-exercise generation and automatic checking** of shell scripts, building on the indexed lab material.
- **Diagnosis beyond the four solver families**, for example memory allocation (first, best and worst fit), address translation and synchronization problems.
- **Validation of the misconception rules and a usability study** with students of the course.
- **Evaluation of answer quality across LLMs**, including a locally hosted model, and safe handling of concurrent sessions in the progress file.
- **Optional deployment with authentication**, if the system is to be used beyond a single local machine.

8. References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459-9474. arXiv:2005.11401.

[3] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009, doi: 10.1561/1500000019.

[4] dorianbrown, "rank_bm25: A collection of BM25 algorithms in Python," GitHub repository. [Online]. Available: https://github.com/dorianbrown/rank_bm25

[5] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," arXiv:2309.07597, 2023.

[6] Beijing Academy of Artificial Intelligence (BAAI), "bge-base-en-v1.5," Hugging Face model card. [Online]. Available: https://huggingface.co/BAAI/bge-base-en-v1.5

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982-3992, doi: 10.18653/v1/D19-1410.

[8] Chroma, "Chroma documentation." [Online]. Available: https://docs.trychroma.com/

[9] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, Boston, MA, USA, 2009, pp. 758-759, doi: 10.1145/1571941.1572114.

[10] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv:2508.10925, 2025.

[11] Groq, "GPT OSS 120B," GroqDocs. [Online]. Available: https://console.groq.com/docs/model/openai/gpt-oss-120b

[12] Streamlit, "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/

[13] Artifex Software, "PyMuPDF documentation." [Online]. Available: https://pymupdf.readthedocs.io/

[14] S. Canny, "python-pptx documentation." [Online]. Available: https://python-pptx.readthedocs.io/

[15] Course lecture slides, Operating Systems, Modules 1-5, Vellore Institute of Technology (course material used as a knowledge-base source).

[16] H.-W. Loidl, "Basic shell scripting using bash," in *Basic Linux Usage* (course notes), Heriot-Watt University. [Online]. Available: https://www.macs.hw.ac.uk/~hwloidl/Courses/LinuxIntro/x695.html